

Assignment Report

Assignment Report _ 2022059952 박준서

Design



Design

Explain your implementation plan to meet the assignment requirements.

주어진 과제 명세에 따라, 구현해야 하는 함수 `ps(int pid)` 는 `pid` 라는 parameter를 받아, 해당 PID를 갖고 있는 process의 `pid`, `state`, `name` 을 출력한 후, Process의 존재 여부 및 올바른 출력 여부에 따라 다른 값을 return함을 확인하였다. 이때 xv6 운영체제는 `ps` command를 입력했을 때 argument로 들어온 모든 Process의 states를 출력해야 하므로, argument의 개수에 따라 `ps` 함수를 얼마나 반복해야 하는지 결정할 수 있다는 점을 알 수 있다.

먼저 process가 어떤 형태로 정의된 구조체인지를 알아보기 위해, `kernel/proc.h`에서 `proc` 구조체 정의를 확인하였다.

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state;           // Process state
    void *chan;                     // If non-zero, sleeping on chan
    int killed;                     // If non-zero, have been killed
    int xstate;                     // Exit status to be returned to parent's wait
    int pid;                        // Process ID

    // wait_lock must be held when using this:
    struct proc *parent;            // Parent process

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;                  // Virtual address of kernel stack
    uint64 sz;                      // Size of process memory (bytes)
    pagetable_t pagetable;          // User page table
    struct trapframe *trapframe;    // data page for trampoline.S
    struct context context;          // swtch() here to run process
    struct file *ofile[NFILE];      // Open files
    struct inode *cwd;               // Current directory
    char name[16];                  // Process name (debugging)
};
```

`kernel/proc.h` 파일에서 확인한 `proc` 구조체와, process의 상태를 표현하기 위한 `procstate` enumeration.

과제에서 명시한 process의 정보를 출력하기 위해서는 이 구조체에서 각각 멤버 변수 `pid`, `state`, `name` 을 출력하면 됨을 확인하였다.

`ps` command를 입력했을 때 모든 active process에 대한 정보를 출력해야 했다. Process Table이 어떤 식으로 구현되어 있는지에 대한 정보를 확인하기 위해 `kernel/proc.c`에 정의되어 있던 `procdump()` 함수를 살펴보았다.

```
662 // Print a process listing to console. For debugging.
663 // Runs when user types ^P on console.
664 // No lock to avoid wedging a stuck machine further.
665 void
666 procdump(void)
667 {
668     static char *states[] = {
669         [UNUSED]    "unused",
670         [USED]       "used",
671         [SLEEPING]   "sleep ",
672         [RUNNABLE]   "runble",
673         [RUNNING]    "run   ",
674         [ZOMBIE]     "zombie"
675     };
676     struct proc *p;
677     char *state;
678
679     printf("\n");
680     for(p = proc; p < &proc[NPROC]; p++){
681         if(p->state == UNUSED)
682             continue;
683         if(p->state >= 0 && p->state < NELEM(states) && states[p->state])
684             state = states[p->state];
685         else
686             state = "???";
687         printf("%d %s %s", p->pid, state, p->name);
688         printf("\n");
689     }
690 }
```

`procdump()` 함수에서 가장 눈여겨보았던 지점은 `proc` 구조체에 대한 pointer `p`를 선언하고, `proc[NPROC]`의 주솟값까지 `p`를 하나씩 증가시키며 process를 탐색하는 부분이었다. (위 스크린샷에서 line 680에 해당) 여여기서 `proc` 배열이 Process Table을 의미함을 확인하고, xv6 운영체제에서 Process Table을 배열의 형태로 관리하고 있음을 파악했다. 따라서 모든 active process 또는 입력으로 들어온 특정 PID를 가지는 process들을 출력할 때, `procdump()`에서 구현한 바와 같이 `proc` 배열을 `proc[0]`부터 순회하면서 입력에서 원하는 조건의 process와 맞는지 체크하면 된다고 생각하였다.

한편 과제 명세에 제시된 System call `ps`는 다음과 같은 signature를 가지며,

```
int ps(int pid);
```

argument로 0이 들어오면 모든 active process를 출력해야 한다 하였으므로 함수 `ps()`는 다음과 같이 구현되어야 한다고 생각했다.

- `pid == 0`인 경우와 `pid != 0`인 경우로 분기한다.
 - `pid == 0`인 경우,
`procdump()` 함수에서 구현한 바와 같이 `proc[0]`부터 순회하며 모든 active process를 출력한다.
 - `pid != 0`인 경우,

Process Table (`proc` 배열)을 `proc[0]` 부터 순회하며 각 entry에 들어있는 process의 PID가 입력으로 들어온 `pid` 와 같은지 비교하여, 입력과 같은 PID를 가진 process를 찾으면 해당 process의 정보를 출력한다. 단, 입력으로 들어온 `pid` 와 같은 PID를 가진 process를 찾지 못한 경우 1을 return한다.

Implementations



Implementation

Detail the specific modifications and additions you made to the given xv6 codebase.

Step 1. `ps()` System Call 구현

`kernel/proc.c` 에 과제 명세에서 주어진 signature에 맞게 `ps()` 함수를 구현하였다.

`ps()` 함수는 앞선 **Design**에서 계획한 바와 같이, argument로 들어온 `pid`의 값이 0인지 아닌지에 따라 다음과 같이 process의 정보를 출력하도록 구현하였다.

- `pid == 0` 인 경우,
`proc.c` 파일에서 주어진 Process Table (`proc[]`)을 0번째 index부터 순회하면서 active한 process (process의 state가 `UNUSED`가 아닌 경우)를 양식에 맞게 출력하도록 했다.
- `pid != 0` 인 경우,
`proc.c` 파일에서 주어진 Process Table (`proc[]`)을 0번째 index부터 순회하면서, 입력값으로 주어진 `pid`와 동일한 PID를 가진 process를 찾으면 그 process의 정보를 양식에 맞게 출력하도록 했다.
만약 Process Table을 끝까지 탐색했는데도 `pid`와 동일한 PID를 가진 process를 찾지 못한 경우 `ps failed` 메시지를 출력하도록 했다.

```

692 // For assignment.
693 // Print all pid, state, name of processes
694 int ps(int pid){
695     char* states[] = {
696         [UNUSED]    "unused",
697         [USED]       "used",
698         [SLEEPING]   "sleep",
699         [RUNNABLE]   "runble",
700         [RUNNING]    "run",
701         [ZOMBIE]     "zombie",
702     };
703
704     struct proc* p;
705     if(pid == 0){ // print informations of all processes.
706         for(p = proc; p < &proc[NPROC]; p++){
707             if(p->state == UNUSED) continue;
708             printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
709         }
710         return 0;
711     }
712     else{
713         // find proc whose PID is pid.
714         for(p = proc; p < &proc[NPROC]; p++){
715             if(p->pid == pid) break;
716         }
717
718         if(p < &proc[NPROC] && p->pid > 0){ // if process exists
719             printf("%d,%s,%s\n", p->pid, states[p->state], p->name);
720             return 0;
721         }
722         else{
723             printf("ps failed : a process of which PID is %d does not exist.\n", pid);
724             return 1;
725         }
726     }
727 }

```

Step 2. kernel/defs.h 에 ps() 함수 선언

kernel/defs.h 에는 kernel/proc.c 에서 구현한 ps() 함수를 추가하여, 이 header file을 include하는 다른 파일에서 ps() 함수를 사용할 수 있도록 하였다. (정다영식 서술이라 조금 수정할 필요 있음)

```

103 void        procdump(void);
104 int         ps(int); // for assignment
105

```

Step 3. kernel/sysproc.c 에 Wrapper Function 구현

```

111 // for assignment.
112 uint64 sys_ps(void){
113     int pid;
114     argint(0, &pid);
115     return ps(pid);
116 }

```

kernel/sysproc.c 에는 ps() 함수를 wrapping하는 Wrapper Function으로서 sys_ps() 함수를 정의하였다. argument 를 받는 ps() 함수를 wrap하기 위해 sys_ps() 함수 내부에서는 argint() 함수를 호출하여, 이후 User program에서 ps

command와 함께 들어오는 argument를 저장하는 Trapframe으로부터 argument를 가져와 `ps()` 함수의 argument로 입력되도록 하였다. (좀 더 매끄럽게 못하나??)

Step 4. `kernel/syscall.h` 및 `kernel/syscall.c` 에 `sys_ps` 선언

```
1 // System call numbers
2 #define SYS_fork 1
3 #define SYS_exit 2
4 #define SYS_wait 3
5 #define SYS_pipe 4
6 #define SYS_read 5
7 #define SYS_kill 6
8 #define SYS_exec 7
9 #define SYS_fstat 8
10 #define SYS_chdir 9
11 #define SYS_dup 10
12 #define SYS_getpid 11
13 #define SYS_sbrk 12
14 #define SYS_pause 13
15 #define SYS_uptime 14
16 #define SYS_open 15
17 #define SYS_write 16
18 #define SYS_mknod 17
19 #define SYS_unlink 18
20 #define SYS_link 19
21 #define SYS_mkdir 20
22 #define SYS_close 21
23 #define SYS_ps 22
```

`kernel/syscall.h`

```
106 // An array mapping syscall numbers from syscall.h
107 // to the function that handles the system call.
108 static uint64 (*syscalls[])(void) = {
109     [SYS_fork] sys_fork,
110     [SYS_exit] sys_exit,
111     [SYS_wait] sys_wait,
112     [SYS_pipe] sys_pipe,
113     [SYS_read] sys_read,
114     [SYS_kill] sys_kill,
115     [SYS_exec] sys_exec,
116     [SYS_fstat] sys_fstat,
117     [SYS_chdir] sys_chdir,
118     [SYS_dup] sys_dup,
119     [SYS_getpid] sys_getpid,
120     [SYS_sbrk] sys_sbrk,
121     [SYS_pause] sys_pause,
122     [SYS_uptime] sys_uptime,
123     [SYS_open] sys_open,
124     [SYS_write] sys_write,
125     [SYS_mknod] sys_mknod,
126     [SYS_unlink] sys_unlink,
127     [SYS_link] sys_link,
128     [SYS_mkdir] sys_mkdir,
129     [SYS_close] sys_close,
130     [SYS_ps] sys_ps,
131 };
```

`kernel/syscall.c`

`kernel/syscall.h` 와 `kernel/syscall.c` 에 `sys_ps`를 선언함으로써 Step 3에서 정의한 Wrapper Function `sys_ps()`가 System call로 등록되도록 하였다.

Step 5. `user/user.h` 에 `ps()` 함수 선언

```
23 int getpid(void);
24 char* sys_sbrk(int,int);
25 int pause(int);
26 int uptime(void);
27 int ps(int);
```

`user/user.h` 에 `ps()` 함수를 선언하여, User Program에서 우리가 만든 System Call을 사용할 수 있도록 하였다.

Step 6. `user/usys.pl` 에 매크로 추가

```
40 entry("dup");
41 entry("getpid");
42 entry("sbrk");
43 entry("pause");
44 entry("uptime");
45 entry("ps");
```

`user/usys.pl` 에 `entry("ps")` 코드를 추가하였다.

`usys.pl` 은 kernel의 System call을 handling하는 Assembly code인 `usys.S`를 생성하도록 도와주는 파일로, 위와 같이 코드를 추가함은 `ps` System call을 User program에서 호출할 수 있도록 하는 매크로를 추가함을 의미한다.

Step 7. user/ps.c 파일 추가

```
1  #include "kernel/types.h"
2  #include "user/user.h"
3
4  int main(int argc, char* argv[]){
5      printf("pid,state,name\n");
6      if(argc < 2){ // print all states of all processes.
7          ps(0);
8      }
9      else{ // print all states of processes whose pid is argv[i]
10         for(int i=1; i<argc; i++){
11             ps(atoi(argv[i]));
12         }
13     }
14     return 0;
15 }
```

새로운 System Call을 이용하는 프로그램인 `user/ps.c` 를 위와 같이 작성하였다.

`ps` command를 입력했을 때 해당 `main()` 함수가 실행되는데, command line으로 우리가 정보를 출력하고자 하는 process들의 PID가 얼마나 들어왔는지에 따라 다르게 실행되므로 `argc` 의 값에 따라 `ps()` 함수를 다르게 실행하도록 하였다.

`argc == 1` 인 경우 command line에는 `ps`만 입력된 것이므로 과제 명세에 따라 argument로 0을 전달하여 `ps()`를 호출하도록 하였고,

`argc >= 2` 인 경우에는 command line에 입력된 PID가 각각 `argv[i]` (`1 <= i < argc`)에 들어있으므로 `argv` 배열을 순회하며 이 PID를 argument로 받는 `ps()` 함수를 호출하도록 하였다.

Step 8. Makefile 수정

```

128 UPROGS=\
129     $U/_cat\
130     $U/_echo\
131     $U/_forktest\
132     $U/_grep\
133     $U/_init\
134     $U/_kill\
135     $U/_ln\
136     $U/_ls\
137     $U/_mkdir\
138     $U/_rm\
139     $U/_sh\
140     $U/_stressfs\
141     $U/_usertests\
142     $U/_grind\
143     $U/_wc\
144     $U/_zombie\
145     $U/_logstress\
146     $U/_forphan\
147     $U/_dorphan\
148     $U/_ps\

```

새로운 사용자 프로그램을 build할 수 있도록 Makefile의 User program section에 `_ps` 를 추가하였다.

Results



Results

Document the compilation and execution process.

Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.

Compilation

```

junseo@bagjunseoul-MacBookAir assignment1-bjsbest % make qemu
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-
strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcpy -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-
builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vprintf -fno-builtin-vprintf -fno-
kernel/proc.o kernel/proc.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T kernel/kernel.ld -o kernel/kernel.elf kernel/entry.o kernel/start.o kernel/console.o kernel/print.o kernel/uart.o kernel/hal.o kernel/spinloc
k.o kernel/string.o kernel/main.o kernel/vm.o kernel/proc.o kernel/watch.o kernel/trampoline.o kernel/trap.o kernel/syscall.o kernel/sysproc.o kernel/bio.o kernel/fs.o kernel/log.o kernel/s
leeplock.o kernel/file.o kernel/pipe.o kernel/exec.o kernel/sysfile.o kernel/kernelvec.o kernel/plio.o kernel/virtio_disk.o
riscv64-unknown-elf-ld: warning: kernel/kernel has a LOAD segment with RWX permissions
riscv64-unknown-elf-objdump -S kernel/kernel > kernel/kernel.asm
riscv64-unknown-elf-objdump -t kernel/kernel | sed '1,/SYMBOL TABLE/d; s/ . * / /; /"/s/d' > kernel/kernel.sym
riscv64-unknown-elf-gcc -Wall -Werror -Wno-unknown-attributes -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -MD -mmodel=medany -ffreestanding -fno-common -nostdlib -fno-builtin-
strncpy -fno-builtin-strncmp -fno-builtin-strlen -fno-builtin-memset -fno-builtin-memmove -fno-builtin-memcpy -fno-builtin-log -fno-builtin-bzero -fno-builtin-strchr -fno-builtin-exit -fno-
builtin-malloc -fno-builtin-putc -fno-builtin-free -fno-builtin-memcpy -Wno-main -fno-builtin-printf -fno-builtin-printf -fno-builtin-vprintf -fno-builtin-vprintf -fno-builtin-vprintf -fno-
-o user/ps.o user/ps.c
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_ps user/ps.o user/ulib.o user/usys.o user/print.o user/umalloc.o
riscv64-unknown-elf-objdump -S user/_ps > user/_ps.asm
riscv64-unknown-elf-objdump -t user/_ps | sed '1,/SYMBOL TABLE/d; s/ . * / /; /"/s/d' > user/_ps.sym
mkfs/mkfs fs.img READING user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/
_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_ps
mmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 997 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virti
o-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
5

```

Execution Process

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ps
pid,state,name
1,sleep,init
2,sleep,sh
3,run,ps
$ ps 1
pid,state,name
1,sleep,init
$ ps 1 2
pid,state,name
1,sleep,init
2,sleep,sh
$ ps 35
pid,state,name
ps failed : a process of which PID is 35 does not exist.
$
```

시스템의 execution flow는 하단 **System Call Execution Flow** 파트에 작성하였다.

Troubleshooting



Troubleshooting

If you encountered any issues during the assignment, describe the issues and your resolution process.

For unresolved issues, explain what they were and what approaches you attempted.

User Program을 통해 System Call을 호출하는 과정을 과제 초반에 잘못 이해해 발생한 2가지 오류가 있었다.

```
junseo@bagjunseoul-MacBookAir assignment1-bjsbest % make qemu
riscv64-unknown-elf-ld -z max-page-size=4096 -T user/user.ld -o user/_ps user/ps.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-unknown-elf-ld: user/usys.o: in function `ps':
/Users/junseo/assignment1-bjsbest/user/usys.S:110: multiple definition of `ps'; user/ps.o:/Users/junseo/assignment1-bjsbest/user/ps.c:11: first defined here
riscv64-unknown-elf-ld: user/ps.o: in function `ps':
/Users/junseo/assignment1-bjsbest/user/ps.c:14:(.text+0xc): undefined reference to `sys_ps'
make: *** [user/_ps] Error 1
```

각 오류가 왜 발생했고 어떻게 해결했는지를 설명하기 전, make qemu를 했을 때 코드가 어떻게 작성되었는지를 먼저 검토해보고자 한다. 대표적으로 살펴봐야 할 점들은 다음과 같았다.

- sysproc.c에서 정의한 `sys_ps()` 함수

```
uint64 sys_ps(int pid){
    return ps(pid);
}
```

- user/ps.c

```
int ps(int pid){
    if(pid >= 0){
        sys_ps(pid);
        return 0;
    }
    else{
```



```

        printf("Invalid input\n");
        return 1;
    }
}

int main(int argc, int* argv){
    if(argc < 2) ps(0);
    else{
        for(int i=1; i<argc; i++){
            ps(atoi(argv[i]));
        }
    }
    return 0;
}

```

위와 같이 코드를 짤 때의 의도는

- ps command를 입력하면 `user/ps.c`의 `main()`이 같은 파일의 `ps()`를 호출하여,
- `ps()` 함수가 kernel의 Wrapper Function을 호출하도록

하는 것이었다. 이렇게 구현한 후 컴파일을 시도했을 때 다음과 같은 두 가지 이슈가 있었다.

Troubleshooting 1. Multiple definition of 'ps' : Linking issue

Multiple Definition error message에 따르면 `user/ps.o`와 `user/usys.o`에서 `ps()`라는 동일한 이름의 함수가 선언돼 에러가 발생했다는 것이었다. `user/usys.pl` 파일을 보면 entry로 넣은 함수에 이름에 맞춰 Assembly Code를 생성하는데, 여기서 생성된 `usys.o` 파일과 `ps.o` 파일을 linking하는 과정에서 동일한 `ps`를 마주치게 돼 error가 발생한 것이 그 이유였다.

이때문에 `ps.c`에서 별도의 `ps()`라는 함수를 선언하면 안됨을 확인했고, 이참에 처음 `ps()`에 구현했던 로직을 `ps.c`의 `main` 함수에 대신 구현하기로 했다.

Troubleshooting 2. Undefined Reference to 'sys_ps'

두 번째 문제는 `user/ps.c`의 `main()`에서 호출한 `sys_ps()`를 찾을 수 없다고 뜬 것이었다. 이 시점에서 `sys_ps()`의 definition은 `kernel/sysproc.c`에 있었고, `syscall.h`와 `syscall.c`를 통해 `sys_ps()` 함수를 System Call로 등록한 상황이었다.

```

// kernel/sysproc.c
int sys_ps(int pid){
    return ps(pid);
}

```

한편 이 시점에서 `syscall.c`에는 `sys_ps` 함수를 다음과 같이 정의하고 있었고,

```

// kernel/syscall.c
extern uint64 sys_ps(void);

```

`user/user.h`에서는 `user`가 호출할 System call로써 다음과 같이 정의하고 있었다.

```

int sys_ps(int);

```

`sys_ps()` 함수의 signature가 통일되지 않은 점은 차치하고, `ps.c` 에서 호출하는 `sys_ps()` 함수를 찾을 수 없다는 점을 이해하지 못했는데, 이것 역시 **Troubleshooting 10**에서와 마찬가지로 Assembly code를 만드는 `usys.pl` 에서 매크로로써 `ps` 만을 entry로 넣어놨기 때문이었다. 즉 Undefined Reference error가 발생한 것은 technical하게는 User program이 `user.h` 에서 선언된 `sys_ps` 함수의 구현부를 찾으려 하였으나, `ps.o` 와 `usys.o` 를 linking하는 과정에서 어떤 코드에도 `sys_ps` 라는 이름의 함수가 구현된 부분을 찾지 못해 발생한 일이라고 풀이된다. 개념적으로는 kernel에서 wrap 하여 얻은 wrapper function을 User program에서 직접 호출하는 것이라고 오해했다.

이 error를 해결하기 위해 고쳐야 하는 점들은 다음과 같았다.

- User Program에서 `ps` 라는 이름의 함수를 호출하고 있으므로 `user.h` 에서 선언한 함수의 이름을 `sys_ps` 에서 `ps` 로 고쳐야 했다.
- 여러 파일에서 선언한 Wrapping function의 signature를 통일해야 했다.

Wrapping function의 signature를 통일하는 과정에서, Wrapping function은 반드시 parameter type을 void로 가져야 함을 확인하였다. 표면적으로는 `syscall.c` 에서 System Call들의 배열이 `int (void)` type의 함수 포인터 배열로 이루어졌기 때문이고, 내부적으로는 User program에서 argument로 들어온 값들은 System call에서도 동일하게 argument로 전달되지 않고 레지스터를 통해 전달되기 때문이었다.

따라서 Wrapping Function `sys_ps()` 를 다음과 같이 `argint()` 를 통해 인자를 전달하는 방식으로 수정하였다.

```
111 // for assignment.
112 uint64 sys_ps(void){
113     int pid;
114     argint(0, &pid);
115     return ps(pid);
116 }
```

System Call Execution Flow



System Call Execution Flow

Describe the step-by-step execution process of the system call, tracing the complete path from the user-level program invocation down to the kernel-level execution and back.

Step 1.

사용자가 shell을 통해 `ps` command를 입력하면, argument의 개수(`argc`)에 따라 `ps.c` 의 `main()` 에서 분기하여 user에서 정의한 `ps()` 함수를 호출한다.

Step 2.

`usys.pl` 에 의해 만들어진 `usys.S` Assembly code를 통해 `ecall` 이 실행되면 `trampoline.S` 의 `uservec` 을 통해 Kernel mode로 진입한다.

Step 3.

Kernel Mode로 진입한 후에는 Interrupt Handler가 `syscall()` 함수를 호출한다.

Step 4.

`syscall()` 함수는 `syscalls[]` 배열을 참조하여 `sysproc.c` 에 정의한 Wrapping Function `sys_ps()` 를 호출하고, `sys_ps()` 내부에서 `proc.c` 에 정의한 `ps()` 함수를 호출한다.

Step 5.

`ps()` 함수의 return값을 Wrapping Function `sys_ps()` 가 return한다. `sys_ps()` 의 이 return값은 `syscall()` 함수에서 `p->trapframe->a0` 으로 명명된 레지스터가 저장한다.

Step 6.

`trampoline.S`의 `userret`을 통해 Kernel mode를 빠져나와 User Mode로 진입한다.